EXPERIMENT 3: GROUP COMMUNICATION

Date Of Performance:
Date Of Submission:

1. Aim:

To study and implement Group Communication where a single sender communicates with multiple receivers using broadcast/multicast communication.

Objectives

1. To understand the concept of group communication in distributed systems.
2. To study how messages can be efficiently transmitted to multiple processes or nodes simultaneously.
3. To implement and analyze basic group communication mechanisms such as broadcast or multicast.
4. To observe how group communication improves communication efficiency compared to individual message passing.
5. To explore its applications in real-world systems such as chat applications, multiplayer games, video conferencing, and distributed databases.

Scope of Client/Server using RPC/RMI

1. Group communication plays an important role in distributed systems where multiple processes or nodes need to exchange information simultaneously. It enables efficient message delivery to a group of participants rather than sending messages individually to each member. This makes communication faster, more scalable, and suitable for large distributed environments.

2. In real-world applications, group communication is widely used in systems such as chat applications, online multiplayer games, and video conferencing platforms, where messages or updates must be delivered to many users at the same time. It is also essential in replicated databases and distributed storage systems, where data updates must be consistently shared among multiple servers to maintain synchronization.

3. The concept can be further expanded by implementing multicast communication, which reduces network traffic by sending data only to members of a specific group. Reliability mechanisms such as message ordering, acknowledgments, and fault tolerance can be integrated to ensure correct and consistent communication even in the presence of network failures.

4. Additionally, security features like encryption, authentication, and access control can be applied to protect group communication from unauthorized access. With the addition of scalability techniques, group communication can support large-scale distributed applications involving thousands or millions of users, making it a fundamental component in modern distributed computing systems.

2. Literature Survey:

2.1 Theory:

2.1 Group Communication

Group Communication is a communication model used in distributed systems where a single process sends a message to multiple processes at the same time. Instead of communicating with each receiver individually, the sender transmits the message to a group, and all members of that group receive the message simultaneously. This approach simplifies communication because the sender does not need to know the identities or addresses of all the receivers. It only needs to send the message to the group, and the underlying system ensures that the message is delivered to all group members.

In distributed environments, many processes run on different machines and must coordinate their actions or share information. Group communication makes this coordination efficient by allowing a single message to reach many processes. This reduces network traffic and processing overhead compared to sending separate messages to each receiver. It also improves scalability when the number of participants in the system increases.

Group communication is commonly implemented using networking techniques such as broadcast and multicast. These techniques allow messages to be transmitted over the network and received by multiple nodes simultaneously. By using protocols and socket programming, systems can efficiently manage communication between different processes.

2.2 Applications of Group Communication

Group communication is widely used in many modern computing applications where multiple users or processes need to share information simultaneously. Some of the common applications include:

1. Distributed Systems

In distributed systems, multiple computers work together to perform tasks. Group communication helps these systems coordinate operations, share updates, and maintain consistency across different nodes.

2. Multiplayer Games

Online multiplayer games rely heavily on group communication to synchronize the game state among players. For example, when one player performs an action, the update must be sent to all other players in the game session.

3. Video Conferencing

Video conferencing platforms require real-time communication between multiple participants. Group communication allows audio, video, and chat messages to be transmitted to all participants efficiently.

4. Replicated Databases

In replicated database systems, copies of the same data are stored on multiple servers. Group communication ensures that updates made to the database are sent to all replicas so that the data remains consistent across all servers.

2.3 Types of Group Communication

Group communication can be categorized into different types depending on how messages are delivered to receivers.

1. Broadcast

Broadcast communication occurs when one sender transmits a message to all nodes in a network. Every node connected to the network receives the message regardless of whether it needs the information or not. Broadcast is simple but may cause unnecessary network traffic when many nodes are involved.

2. Multicast

Multicast communication allows a sender to transmit a message only to a selected group of receivers. Only the nodes that belong to the multicast group will receive the message. This method is more efficient than broadcast because it reduces unnecessary data transmission.

3.Anycast

Anycast communication allows a sender to send a message to any one member of a group rather than all members. The network automatically selects one receiver, usually the nearest or most available node, to handle the request. This technique is often used in distributed services for load balancing and efficient resource utilization.

2.4 System Requirements

To implement group communication using socket programming, certain system requirements are necessary:

Operating System:

The experiment can be performed on operating systems such as Linux or Windows, both of which support socket programming and networking features.

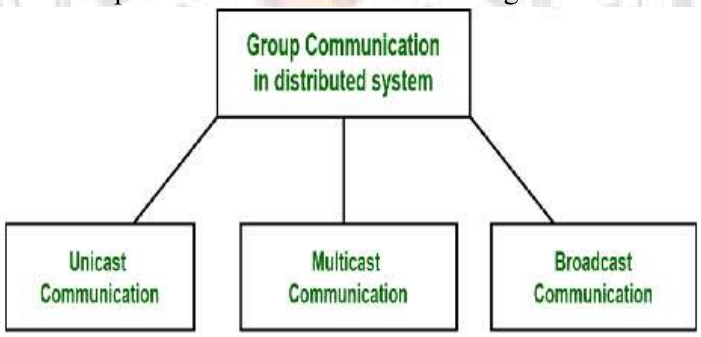
Programming Language:

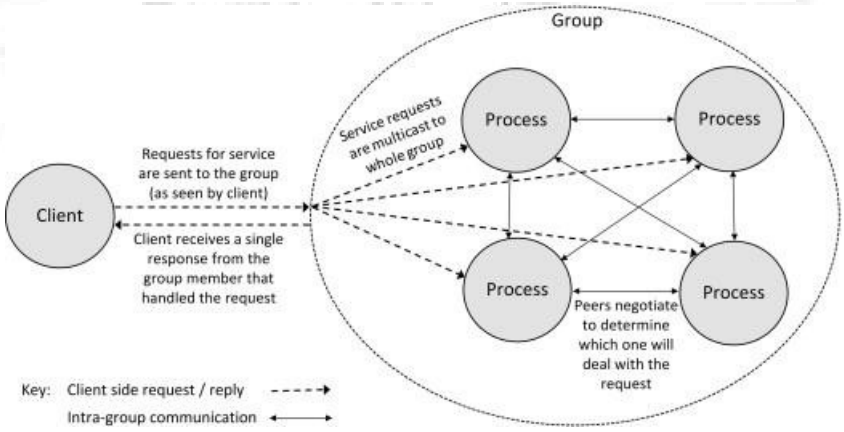
Python is used as the programming language because it provides simple libraries for network communication and socket programming.

Network:

The communication is performed on a Localhost network using UDP sockets. Localhost allows processes running on the same machine to communicate with each other through network protocols without requiring an external network connection.

UDP (User Datagram Protocol) is used because it supports fast communication and allows broadcasting messages to multiple receivers without establishing a connection.





2.5 Algorithm

Group Communication (Broadcast using UDP)

INPUT: Message *Msg* from Sender

OUTPUT: Message displayed by all Receiver processes

BEGIN

SENDER SIDE

1. Create UDP socket **S**
2. Enable broadcast option on socket **S**
3. Define broadcast address **B** and **Port_Number**
4. Enter message **Msg** to be sent to the group
5. Encode message
   M ← Encode(Msg)
6. Send message to broadcast address
   Send M to (B, Port_Number)

RECEIVER SIDE

7. Create UDP socket **R**
8. Bind socket **R** to **Port_Number**
9. Wait for incoming broadcast message

MESSAGE DELIVERY

10. When message **M** is received
    Msg ← Decode(M)
11. Display received message on receiver screen
12. Continue listening for next messages
    TERMINATION
13. If communication ends, close receiver socket **R**
14. Close sender socket **S**
    END

2.2.2 Group Communication (Multicast Concept)

INPUT: Multicast message *Msg*

OUTPUT: Message received by all members of the multicast group

BEGIN

SENDER SIDE

1. Create UDP socket **S**
2. Define multicast group address **G** and **Port_Number**
3. Enter message **Msg**
4. Encode message
   M ← Encode(Msg)
5. Send **M** to multicast group **(G, Port_Number)**

RECEIVER SIDE

6. Create UDP socket **R**
7. Bind socket **R** to **Port_Number**
8. Join multicast group **G**
9. Wait for incoming multicast messages

MESSAGE PROCESSING

10. When message **M** is received
    Msg ← Decode(M)
11. Display received message

TERMINATION

12. Leave multicast group **G**
13. Close receiver socket **R**
14. Close sender socket **S**

END

2.2 Programming Language used:

2.2.1.1 Libraries Used

1. built-in Python functions

Purpose:
Used for basic operations such as printing messages and receiving input from the user.

```
Syntax: print("message")
  input("Enter value: ")
```

Usage in Program:
Used to display menu options, show process activities, and take user input for operations such as creating processes, joining groups, and broadcasting messages.

2. Object-Oriented Programming (Classes)

Purpose:
Python classes allow the program to represent real-world entities such as **processes and groups** in distributed communication.

```
Syntax:
  class ClassName:
  def __init__(self):
  pass
```

Usage in Program:
Used to define the **Process**, **Group**, and **GroupManager** classes which control the entire group communication system.

2.2.1.2 Classes / Methods Used

1. init()
Purpose:
Constructor method used to initialize objects when they are created.
Syntax:
def __init__(self, parameters):
Usage in Program:
Initializes attributes such as process ID, group name, members list, and coordinator.
2. receive()
Purpose:
Allows a process to receive and display messages sent by another process.
Syntax:
def receive(self, msg):
Usage in Program:
Used to display broadcast messages received by group members.

3. join()

Purpose:
Allows a process to join a group.
Syntax:
def join(self, process):
Usage in Program:
Adds the process to the group's member list.

4. leave()
Purpose:
Removes a process from the group.
Syntax:
def leave(self, pid):
Usage in Program:
Deletes the process from the group if it exists.
5. broadcast()
Purpose:
Sends a message from one process to all members of the group.
Syntax:
def broadcast(self, sender, message):
Usage in Program:
Iterates through all group members and sends the message to each process.
6. coordinator_crash()
Purpose:
Simulates the failure of the group coordinator and elects a new coordinator.
Syntax:
def coordinator_crash(self):
Usage in Program:
Selects a new coordinator from the remaining group members.
7. create_process()
Purpose:
Creates a new process in the system.
Syntax:
def create_process(self, pid):
Usage in Program:
Adds a new process object to the process list.
8. create_group()
Purpose:
Creates a new communication group.
Syntax:
def create_group(self, name):
Usage in Program:
Initializes a group where processes can join and communicate.
9. delete_group()
Purpose:
Deletes an existing group.
Syntax:
def delete_group(self, name):
Usage in Program:
Removes the group and its members from the system.
10. input()
Purpose:
Takes user input from the keyboard.
Syntax:
input("Enter value: ")

Usage in Program:
Used in the menu to take user choices and process IDs.
11. print()
Purpose:
Displays output messages on the console.
Syntax:
print("message")

2.3 Program :

2.3.1 Group Communication Using UDP Broadcast (Python)

Process Class

```
class Process:
    def __init__(self, pid):
        self.pid = pid

    def receive(self, msg):
        print(f"Process {self.pid} received: {msg}")
```

Group Class

```
class Group:
    def __init__(self, name, coordinator=None):
        self.name = name
        self.members = {}
        self.coordinator = coordinator

    def join(self, process):
        self.members[process.pid] = process
        print(f"Process {process.pid} joined group '{self.name}'")

    def leave(self, pid):
        if pid in self.members:
            del self.members[pid]
            print(f"Process {pid} left group '{self.name}'")
        else:
            print("Process not in group")

    def broadcast(self, sender, message):
        print(f"\nBroadcast from Process {sender} to Group '{self.name}'")
        for p in self.members.values():
            p.receive(message)

    def coordinator_crash(self):
        print("\nCoordinator crashed!")
        if self.members:
            self.coordinator = next(iter(self.members))
            print(f"New Coordinator elected: Process {self.coordinator}")
```

```
        else:
            print("No members left in group")
```

Group Manager

```
class GroupManager:
    def __init__(self):
        self.groups = {}
        self.processes = {}

    def create_process(self, pid):
        self.processes[pid] = Process(pid)
        print(f"Process {pid} created")

    def create_group(self, name, hierarchical=False):
        coord = None
        if hierarchical:
            coord = int(input("Enter coordinator process ID: "))
        self.groups[name] = Group(name, coord)
        print(f"Group '{name}' created")

    def delete_group(self, name):
        if name in self.groups:
            del self.groups[name]
            print(f"Group '{name}' deleted")
        else:
            print("Group not found")
```

MAIN MENU

```
gm = GroupManager()

while True:
    print("\n==== GROUP COMMUNICATION MENU ====")
    print("1. Create Process")
    print("2. Create Flat Group")
    print("3. Create Hierarchical Group")
    print("4. Join Group")
    print("5. Leave Group")
    print("6. Broadcast Message")
    print("7. Coordinator Crash")
    print("8. Delete Group")
    print("9. Exit")

    choice = input("Enter choice: ")

    if choice == "1":
        pid = int(input("Enter process ID: "))
        gm.create_process(pid)
```

```
elif choice == "2":
    name = input("Enter group name: ")
    gm.create_group(name)

elif choice == "3":
    name = input("Enter group name: ")
    gm.create_group(name, hierarchical=True)

elif choice == "4":
    pid = int(input("Enter process ID: "))
    gname = input("Enter group name: ")
    gm.groups[gname].join(gm.processes[pid])

elif choice == "5":
    pid = int(input("Enter process ID: "))
    gname = input("Enter group name: ")
    gm.groups[gname].leave(pid)

elif choice == "6":
    sender = int(input("Sender process ID: "))
    gname = input("Enter group name: ")
    msg = input("Enter message: ")
    gm.groups[gname].broadcast(sender, msg)

elif choice == "7":
    gname = input("Enter group name: ")
    gm.groups[gname].coordinator_crash()

elif choice == "8":
    gname = input("Enter group name: ")
    gm.delete_group(gname)

elif choice == "9":
    print("Exiting Group Communication")
    break

else:
   print("Invalid choice")
```

Outputs:

```
==== GROUP COMMUNICATION MENU ====
1. Create Process
2. Create Flat Group
3. Create Hierarchical Group
4. Join Group
5. Leave Group
6. Broadcast Message
7. Coordinator Crash
8. Delete Group
9. Exit
Process 1 created

==== GROUP COMMUNICATION MENU ====
1. Create Process
2. Create Flat Group
3. Create Hierarchical Group
4. Join Group
5. Leave Group
6. Broadcast Message
7. Coordinator Crash
8. Delete Group
9. Exit
Group 'vaisak' created
```

```
==== GROUP COMMUNICATION MENU ====
1. Create Process
2. Create Flat Group
3. Create Hierarchical Group
4. Join Group
5. Leave Group
6. Broadcast Message
7. Coordinator Crash
8. Delete Group
9. Exit

Broadcast from Process 1 to Group 'vaisak'
```

```
==== GROUP COMMUNICATION MENU ====
1. Create Process
2. Create Flat Group
3. Create Hierarchical Group
4. Join Group
5. Leave Group
6. Broadcast Message
7. Coordinator Crash
8. Delete Group
9. Exit
Enter choice: 5
Enter process ID: 1
Enter group name: vaisak
Process 1 left group 'vaisak'
```

Conclusion

Group communication is a fundamental concept in distributed systems that enables efficient and coordinated interaction among multiple processes or nodes. Instead of sending messages individually, it allows a single message to be delivered to multiple members of a group, improving communication efficiency and reducing network overhead.

Through this concept, systems can support collaborative and real-time applications such as chat platforms, multiplayer games, video conferencing, and distributed databases. By incorporating mechanisms like multicast communication, reliability, fault tolerance, and security, group communication can ensure consistent, secure, and dependable data exchange among participants.

Sign and Remark:

R1 (4 Marks)	R2 (4 Marks)	R3 (4 Marks)	R4 (3 Marks)	Total Marks (15 Marks)	Signature